

Shallow Copy & Deep Copy

No. / /
Dt: / /

→ Deep Copy :- Value of the new variable disconnected from original variable. It assign diff memory allocation of all the members of X to Y. If X vanishes, Y still in the memory.

→ Shallow Copy :- Some values are still connected to the original variable. It only copies the elements which are present at first level, not deeply nested.

④ Different ways of clone :-

① Using Assignment Operator (=) - use in primitive data type / Deep Copy (primitive) / Shallow Copy

② Using Spread Syntax (...) - Shallow Copy / Deep Copy

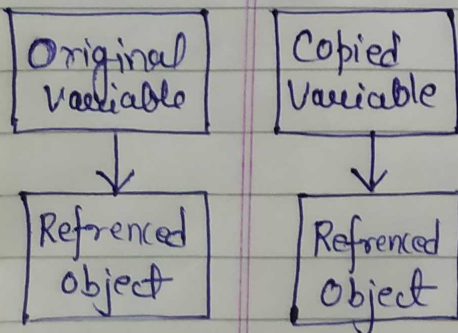
③ Using Object.assign() method - Shallow Copy

④ Using JSON.stringify & JSON.parse() - Deep Copy
Convert object to string parsing & return object

⑤ Using structuredClone() - Deep Copy

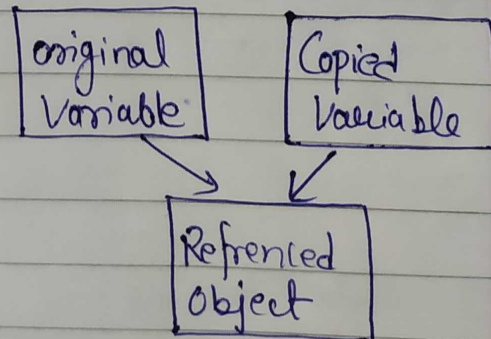
⑥ Using lodash: cloneDeep() method - Deep clone / shallow

④ Deep Copy



Refer to diff references,
changes made in one doesn't
have affect on another.

Shallow Copy



Refer to Same object,
changes made in one
affect other.

* Examples -

① Assignment Operator -

For primitive

```
let counter = 1;  
let CopyCounter = counter;  
CopyCounter = 2;  
console.log(counter); // output: 1
```

For Primitive, can quickly copy value in separate memory space. CONST can't be use here.

For Non-primitive

```
let person = {  
  firstName: 'priya',  
  lastName: 'Supriya',  
};  
let copyPerson = person;  
copyPerson.firstName = 'Jerry';  
console.log(person);  
// { firstName: 'Jerry',  
  lastName: 'supriya' }
```

It will not copy the value. It reference the same object in memory

② Spread Operator -

↳ Deep Copy: If the data is not nested.

↳ Shallow Copy: For array or object create deep copy for top most data & shallow copy of the nested data.

For Object:-

```
const user = {  
  firstName: 'priya',  
  address: {  
    city: 'Mumbai'  
  }  
};
```

```
const copyUser = {...user};  
copyUser.firstName = 'John';  
copyUser.address.city = 'Pune';  
console.log('user');
```

Output :-

```
{ firstName: 'priya', // Deep copy  
  address: {  
    city: 'Pune' // Shallow copy  
  }  
}
```

For Array -

```
const arr = [0, 1, 2, [3, 4, [5]]]
```

```
const copyArr = [...arr]
```

```
copyArr[0] = 10
```

```
copyArr[1] = 11
```

```
copyArr[2] = 12
```

```
copyArr[3] = 13
```

```
copyArr[3] = [...arr[3]]
```

```
console.log(copyArr); // 10, 11, 12, 13
```

```
// [10, 11, 12, [3, 4, [5]]]
```

```
console.log(arr); // [0, 1, 2, [3, 4, [5]]]
```

③ Object Assign:-

```
let person = {
```

```
  firstName: 'priya',
```

```
  address: {
```

```
    city: 'Mumbai'
```

```
  }
```

```
let copyPerson = Object.assign({}, person)
```

```
copyPerson.firstName = 'Joh' // Disconnected // Deep
```

```
copyPerson.address.city = 'Pune' // Connected // Shallow
```

```
console.log(person, copyPerson)
```

```
{  
  firstName: 'priya'
```

```
  address: {
```

```
    city: 'puno'
```

```
  }
```

```
}
```

```
{  
  firstName: Joh
```

```
  address: {
```

```
    city: 'Pune'
```

```
  }
```

```
}
```

- firstName did not change because it's primitive
- Address change because both objects (person & copyPerson) reference the same object (i.e. address).

④ using JSON: -

```

let person = {
  firstName: 'Priya',
  address: {
    city: 'Mumbai'
  }
}

let copyPerson = JSON.parse(JSON.stringify(person))
console.log(person);

```

output: { firstName: 'Priya',
address: {
city: 'Mumbai'}

CopyPerson.firstName = 'John';
CopyPerson.address.city = 'Pune'

⑤ using structuredClone(); - Rather than cloning, transferring the original value. Transferable objects are detached from the original object & attached to the new object.

```

const user = {
  firstName: 'Priya',
  address: { city: 'Mumbai' }
}

const copy cloneUser = structuredClone(user);
cloneUser.firstName = 'John';
cloneUser.address.city = 'Pune';

console.log(user);
{
  firstName: 'Priya',
  address: {
    city: 'Mumbai'
  }
}

console.log(cloneUser);
{
  firstName: 'John',
  address: {
    city: 'Pune'
  }
}

```

⑥ Using Lodash:- It's an external library which allows Shallow Copy (`.clone`) & deep Copy (`.cloneDeep`).

→ Installation Required: `npm i --save lodash`

→ Import Required:- `const _ = require('lodash');`
`import _ from "lodash";`

```
const user = {  
  firstName: 'priya',  
  address: { city: 'Mumbai' }  
}
```

```
const shallowClone = _.clone(user)  
shallowClone.firstName = 'dome'  
shallowClone.address.city = 'Pune'
```

```
{ firstName: 'dome',  
  address: { city: 'mumbai' }  
}
```

```
const deepClone = _.cloneDeep(user);  
deepClone.firstName = 'dome'  
deepClone.address.city = 'Pune'
```

```
{ firstName: 'priya',  
  address: { city: 'Pune' }  
}
```